

Strong and Weak Assemblies in .NET Framework

In this article, I am going to discuss **Strong and Weak Assemblies in .NET Framework** with Examples. Please read our previous article where we discussed [App Domain in .NET](#) Applications. In .NET Framework, the assemblies are broadly classified into 2 types. They are as follows:

1. **Weak Named Assemblies**
2. **Strong Named Assemblies**

Let us first understand what assembly is and then we will discuss strong and weak assembly and the difference between them.

Understanding Assembly in .NET Framework:

Let us create a simple console application with the name **AssemblyDemo** and then modify the **Program** class as shown below. This is a very simple C# program, simply printing a message “**Hello world**” to the console. In order to print the message on the Console, here we are using the **Console** class. That **Console** class is coming from the **System** namespace. And the **System** namespace is present in the **System** Assembly. The System assembly is a .NET Framework assembly

```
using System;
namespace AssemblyDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello world");
            Console.ReadKey();
        }
    }
}
```

When we installed .NET Framework on our machine, two important components get installed. One is the .NET Framework Base Class Library (BCL) and the other one is CLR which is nothing but the Runtime Environment where our .NET Application is going to run. In .NET Framework Base Class Library, we have several assemblies. All the .NET Framework assemblies are installed in a special location called GAC (Global Assembly Cache).

Starting with the .NET Framework 4, the default location for the Global Assembly Cache is `%windir%\Microsoft.NET\assembly`. In earlier versions of the .NET Framework, the default location is `%windir%\assembly`. Once you go to the GAC location, then you will find all the .NET Framework assemblies. We will discuss GAC in detail in our upcoming article. To see the installed assemblies we need to open Visual Studio Command Prompt and we need to type **gacutil -l** or **gacutil /l** command and press the enter button as shown in the below image.

```
Developer Command Prompt x + v
C:\Program Files (x86)\Microsoft Visual Studio\2017\Community>gacutil /l
Microsoft (R) .NET Global Assembly Cache Utility. Version 4.0.30319.0
Copyright (c) Microsoft Corporation. All rights reserved.

Name Version Culture PublicKeyToken Processor Architecture
The Global Assembly Cache contains the following assemblies:
Microsoft.Ink, Version=6.1.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35, processorArchitecture=AMD64
Microsoft.Interop.Security.AzRoles, Version=2.0.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35, processorArchitecture=AMD64
mswacdm, Version=1.8.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35, processorArchitecture=AMD64
Microsoft.Ink, Version=6.1.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35, processorArchitecture=x86
Microsoft.Interop.Security.AzRoles, Version=2.0.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35, processorArchitecture=x86
mswacdm, Version=1.8.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35, processorArchitecture=x86
Microsoft.Ink.Resources, Version=6.1.0.0, Culture=en, PublicKeyToken=31bf3856ad364e35, processorArchitecture=MSIL
Microsoft.ManagementConsole, Version=3.0.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35, processorArchitecture=MSIL
Microsoft.ManagementConsole.Resources, Version=3.0.0.0, Culture=en, PublicKeyToken=31bf3856ad364e35, processorArchitecture=MSIL
Microsoft.Office.Interop.Access, Version=15.0.0.0, Culture=neutral, PublicKeyToken=71e9bce111e9429c, processorArchitecture=MSIL
Microsoft.Office.Interop.Access.Dao, Version=15.0.0.0, Culture=neutral, PublicKeyToken=71e9bce111e9429c, processorArchitecture=MSIL
Microsoft.Office.Interop.Excel, Version=15.0.0.0, Culture=neutral, PublicKeyToken=71e9bce111e9429c, processorArchitecture=MSIL
Microsoft.Office.Interop.Graph, Version=15.0.0.0, Culture=neutral, PublicKeyToken=71e9bce111e9429c, processorArchitecture=MSIL
Microsoft.Office.Interop.OneNote, Version=12.0.0.0, Culture=neutral, PublicKeyToken=71e9bce111e9429c, processorArchitecture=MSIL
Microsoft.Office.Interop.OneNote, Version=15.0.0.0, Culture=neutral, PublicKeyToken=71e9bce111e9429c, processorArchitecture=MSIL
Microsoft.Office.Interop.Outlook, Version=15.0.0.0, Culture=neutral, PublicKeyToken=71e9bce111e9429c, processorArchitecture=MSIL
```

All the assemblies present in GAC are strongly typed. Later part of this article, we will discuss what exactly a strong type assembly is and the difference between a weak and a strong type assembly in the .NET Framework. In .NET Framework, an assembly consists of 4 Parts. They are as follows:

- 1. Simple Textual Name (i.e. Assembly Name).
- 2. The Version Number.
- 3. Cultural information (If provided, otherwise the assembly is language-neutral)
- 4. Public Key Token

Let us discuss each part of an assembly in detail.

Assembly Name (Simple Textual Name):

This is nothing but the project name. We have created one console application with the name **AssemblyDemo**. Now build the project and go to the **Bin => Debug** folder of your project and you should find an assembly with the name **AssemblyDemo**.

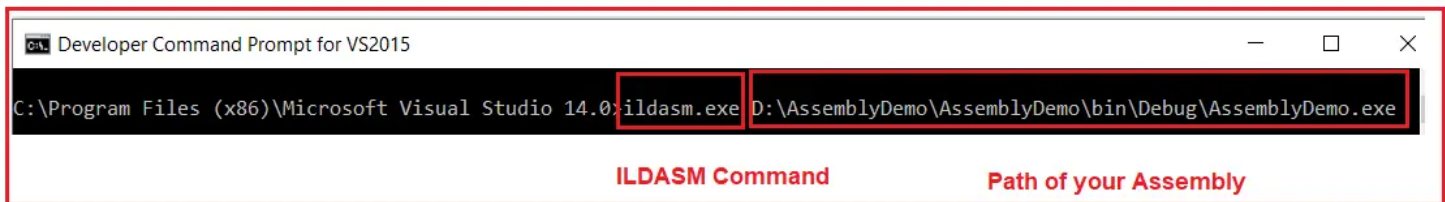
Version Number:

The default format of the Version number is **1.0.0.0**. That means the version number again consists of four parts as follows:

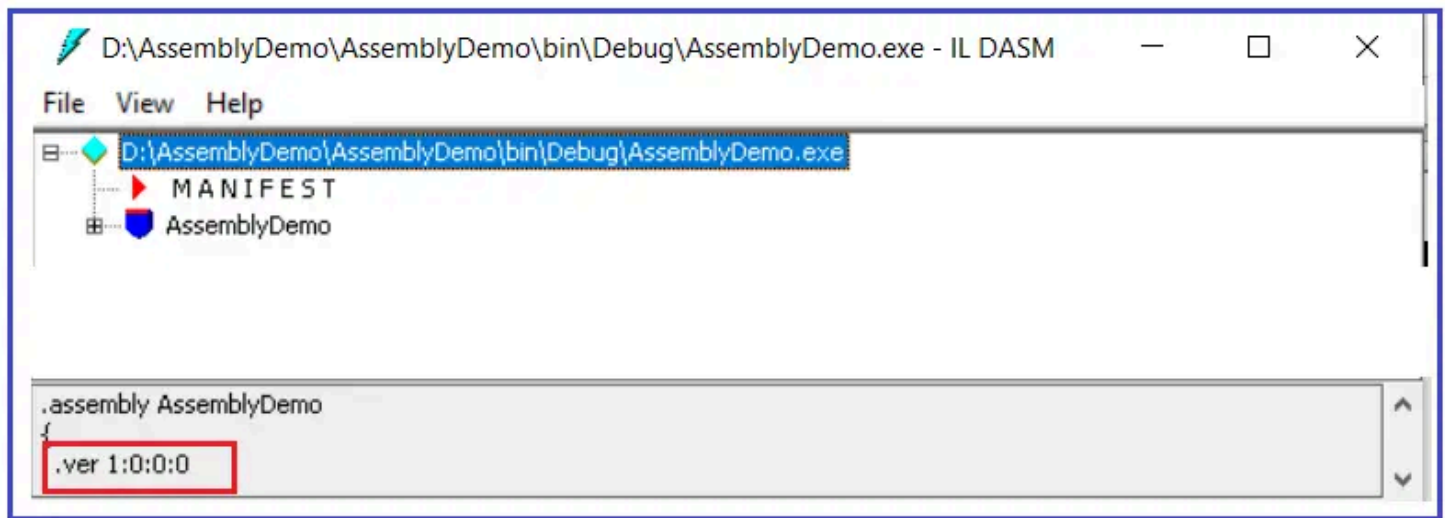
- 1. Major Version
- 2. Minor Version
- 3. Build Number
- 4. Revision Number

Typically, any software we develop will go under changes over a period of time. When we fix bugs or add new features, depending on the significance of the change, we either change the major number or the minor number. If the changes we are making to the application are huge, then probably we change the major number else we will change the minor number. Most of the time the build number and revision number have defaulted.

In order to see the Version Number of our assembly **AssemblyDemo**, Open the Visual Studio Developer Command Prompt and use the **ILDASM** command to see the version number as shown below.



Once you use the ILDASM command followed by the Physical Path of your assembly and hit the enter key, you will get the following ILDASM window and just look at the version number which you can find at the bottom of the window.



Note: You can also check the Version Number of your Project using the AssemblyInfo.cs class file which is present inside the Properties folder of your project. If you open AssemblyInfo.cs class file, then you should see the following AssemblyVersion statement.

[assembly: AssemblyVersion("1.0.0.0")]

How to Change the Version Number of an Assembly in .NET Framework?

If you want to change the Version Number of your assembly, then you need to use the **AssemblyVersion** attribute within the **AssemblyInfo** class which is present inside the **Properties** folder of your project. You can specify all the values or you can default the Revision and Build Numbers by using the '*'. Suppose, you want to change the Major Number to 3 and the Minor Number to 2, then you need to update the following **AssemblyVersion** attribute of the **AssemblyInfo** class as follows.

[assembly: AssemblyVersion("3.2.*")]

With the above changes in place, now if you build the solution and check the version number using the ILDASM tool, then you should see the updated version number. Please read our [ILDASM and ILASM articles](#) to learn more about ILDASM and ILASM.

Assembly Culture:

The AssemblyCulture attribute is used for specifying the culture of the assembly. By default in the .NET Framework assemblies are Language-Neutral which means the AssemblyCulture attribute contains

Culture=neutral. If you go to the GAC, then you will find most of the assemblies are Culture-neutral. But there could be some assemblies that are culture-specific. For a better understanding, please have a look at the following image which you can also find in your GAC. The following assemblies are specific to the language specified in the Culture attribute.

Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=cs	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=de	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=es	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=fr	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=it	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=ja	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=ko	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=pl	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=pt-BR	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=ru	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=tr	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=zh-HANS	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=zh-HANT	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL

When you specify the culture, then that assembly becomes a **Satellite Assembly**. A satellite assembly is a .NET Framework assembly containing resources specific to a given language. Using satellite assemblies, you can place resources for different languages in different assemblies, and the correct assembly is loaded into memory only if the user selects to view the application in that language

If you want to specify the culture then you need to use the **AssemblyCulture** attribute with the **AssemblyInfo.cs** class file. For example, if you want to specify English as the culture then you need to update the AssemblyCulture attribute as shown below.

[assembly: AssemblyCulture("en")]

Public Key Token:

If you go to the GAC, then you will see that every assembly is assigned a public key token as shown in the below image.

Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=cs	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=de	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=es	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=fr	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=it	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=ja	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=ko	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=pl	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=pt-BR	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=ru	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=tr	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=zh-HANS	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL
Microsoft.Web.Administration.resources	Version=7.9.0.0	Culture=zh-HANT	PublicKeyToken=31bf3856ad364e35	processorArchitecture=MSIL

In order to get the Public Key Token, you need to sign your assembly with a Private and Public Key Pair. Now the question is how do I get the Private-Public Key?

How do I get the Private-Public Key?

In the .NET Framework, we have a tool called Strong Naming Tool (sn.exe) and we can use this (sn.exe) tool to generate the Private Public Key pair. Again in order to use this tool, we need to use the Developer Command Prompt for Visual Studio. So, open Developer Command Prompt for Visual Studio in Administrator mode and then type **sn.exe -k D:\MyKeyFile.snk** and press enter button as shown in the below image.

```
Developer Command Prompt  X  +  v

C:\Program Files\Microsoft Visual Studio\2022\Community>sn.exe -k D:\MyKeyFile.snk

Microsoft (R) .NET Framework Strong Name Utility  Version 4.0.30319.0
Copyright (c) Microsoft Corporation.  All rights reserved.

Key pair written to D:\MyKeyFile.snk
```

Once you type the required command and press enter, the key file with the name **MyKeyFile.snk** should be generated in the D: Drive. In SN.exe, SN stands for Strong Name.

Once you generated the Key file, then you need to use the **AssemblyKeyFile** attribute in the **AssemblyInfo** class to sign the assembly with a strong name. To the constructor of the AssemblyKeyFile attribute, you need to pass the path of the key file that contains the private and public keys as shown below.

[assembly: AssemblyKeyFile("D:\\MyKeyFile.snk")]

Once you add the above AssemblyKeyFile attribute, build the solution. Once you build the solution, now your assembly sign with a private-public key pair. Now, our assembly has all four components i.e. Name, Version Number, Culture, and Public Key Token. To see the assembly information, please modify the Program class as follows.

```
using System;
using System.Reflection;
namespace AssemblyDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Replace D:\Projects\AssemblyDemo\AssemblyDemo\bin\Debug\Assembly
            //With your Assembly Path

            Console.WriteLine(Assembly.LoadFile(@"D:\Projects\AssemblyDemo\AssemblyDemo\bin\
            Console.ReadKey();
        }
    }
}
```

Now, run the application and you should see the following result.

```
D:\Projects\AssemblyDemo\A  X  +  v

AssemblyDemo, Version=1.0.0.0, Culture=neutral, PublicKeyToken=0b27bee49db7bd32
```

Strong Name Assembly in .NET Framework:

An assembly is said to be strongly named assembly when it has the following properties

1. **The Assembly Name.**
2. **Version Number.**
3. **The Assembly should have been signed with the Private/Public Key Pair.**

What is the Difference Between Strong and Weak Assemblies in .NET Framework?

If an assembly is not signed with the private/public key pair then the assembly is said to be a weak named assembly and it is not guaranteed to be unique and may cause the DLL Hell Problem. The Strong Named Assemblies are guaranteed to be unique and solve the DLL Hell Problem. Again, you cannot install an assembly into GAC unless the assembly is strongly named.

Note: In our upcoming articles, we will discuss what is DLL Hell Problem and how we can resolve the DLL Hell Problem in .NET Framework using Strong Name Assembly with Examples.

In the next article, I am going to discuss GAC in Detail as well as we will discuss [How to Install a Strong Name Assembly into GAC in .NET Framework](#). Here, in this article, I try to explain what are the **Strong and Weak Assemblies in .NET Framework** as well as the different properties of an assembly in detail. I hope you enjoy this Strong and Weak assembly in the .NET article.